

BEFORE YOU TOUCH ANY PRODUCTION DATA

1. **Export a backup** in phpMyAdmin: click the database → **Export** → **Quick** → format **SQL** → **Go**. The file downloads to your PC — that is your safety copy.
2. Know which database your site uses: `config/secrets.local.php` (Laragon) or `config/secrets.live.php` (hosting) → key `db.name` .
3. Remember: **file uploads** (vehicle scans, customer ID copies, part photos) live in the `uploads/` folder on the server — they are **not** inside a normal `.sql` export. Full “replace DB” does not replace those files unless you copy them separately.

Pick one path below. Do *not* mix them in one go (e.g. do not run a full import on top of a half-populated database without a plan).

PATH 1 · REPLACE THE ENTIRE DATABASE (FULL RESET)

Use when: you intentionally want this MySQL database to become an exact copy of another environment (e.g. your Laragon export), or you are wiping a test server and starting clean. **Everything in that database is replaced** — all customers, invoices, parts rows, users, etc., unless you restore from a backup later.

1A · What “replace” means

- After success, **every table** in that database matches what is in your import file (or what the script set created).
- Anything that existed **only** on the old live database and was **not** in the import is **gone** (unless you kept a backup .sql).

1B · Steps (phpMyAdmin, typical hosting)

1. **Backup:** Export the current database to .sql (see above). Store it safely.
2. **Target must be empty:**
 - **Option A — new database:** cPanel → **MySQL® Databases** → create a new database → add your MySQL user with **All Privileges**. In phpMyAdmin, select that **new empty** database.
 - **Option B — same name:** select the database → select **all tables** → **Drop** (or drop one by one if the host requires). Refresh until there are **no tables**.
3. **Import:** With the empty database selected → tab **Import** → choose your .sql file → **Go**.
4. If the import stops with **#1050 Table already exists**, you are **not** empty — go back and drop tables or use a new empty database.
5. If the import stops with **#1227 / DEFINER / root@localhost** on views, the dump was exported from Laragon as root . Strip `DEFINE R= `root`@`localhost`` from the file (or use a cleaned copy); remove phpMyAdmin “stand-in” `CREATE TABLE epc_full_view` if your project doc gives a fixed import file.
6. **Point the app:** if you created a **new** database name, edit **only** `db.name` in `secrets.live.php` on the server — do not commit that file to Git.
7. **Uploads:** copy `uploads/` from the machine that has the scans (e.g. PC → live FTP) if you need those files to work.
8. **Test:** open the site → login (`users` table came from import) → open Customers and one invoice.

1C · Replace using project sql/ scripts only (no giant dump)

On a **brand-new empty database**, you can instead run Autowagen’s numbered scripts from the repo in order (local path example: `sql/`), e.g. `01_auth.sql` , `02_epc.sql` , ... through `07_credit_notes.sql` , skipping any you already merged by hand. After that you must add a **user** in `users` (see [add_users_staff_guide_print.html](#)) and seed data yourself. This path is common for **clean installs**, not for “copy my full dev data” unless you also import data dumps for specific tables.

PATH 2 · UPDATE SCHEMA WHILE KEEPING EXISTING DATA

Use when: live (or staging) already has **real customers, vehicles, invoices, parts**, and you shipped **new PHP** that expects **new columns** or **new tables**. You want to **not** delete business rows.

2A · Core idea

- Do **not** import a **full** database export from another machine over that database — the file usually starts with `CREATE TABLE customers`, which fails with **table already exists**, or it would overwrite data if you forced it.
- Instead, run **incremental migration scripts** from the Autowagen project’s `sql/` folder: they are written to be **additive** — `CREATE TABLE IF NOT EXISTS`, and column changes use guarded `ALTER` patterns where possible.
- Each script should be run **at most once per database** (or be idempotent). If MySQL reports “duplicate column” or “already exists”, note it and skip repeating that block — the structure may already match.

2B · Standard run order (new server or “catch-up” checklist)

Run in phpMyAdmin → select **your** database → **SQL** tab → paste one file at a time → **Go**. **Skip** scripts that were already applied on that database.

Order	File (examples)	Typical adds
1	01_auth.sql	users , login attempts
2	02_epc.sql	EPC catalogue tables + view
3	03_master_data.sql	Vehicles, customers, suppliers, links
4	03b_vehicle_extras.sql ... 03d_customer_compliance.sql	Extra vehicle / customer columns
5	04_inventory.sql + 04b ... 04d	Parts, TPP, purchases, AP payments
6	05_pos.sql	Invoices, lines, payments
7	06a_customer_account.sql	Account customer flags, credit limit
8	06b_web_shop.sql	Shop orders, list_online
9	06e_shop_guest_enquiries.sql	Guest enquiry table (<i>after</i> 06b)
10	07_credit_notes.sql	Credit notes (<i>after</i> 05)

Optional EPC expansion scripts (02b ... 02h) follow the separate order in `sql/README_EPC_EXPANSION.txt` . They add catalogue depth, not customer rows.

2C · How this keeps customer data

- Scripts like `06a_customer_account.sql` typically **ADD COLUMN** to `customers` — existing customer names and IDs stay; new fields get defaults or NULL.
- New tables (`sales_credit_notes` , `shop_orders` , etc.) sit **beside** old data — they do not erase `customers` or `sales_invoices` unless you run a destructive command (do not).
- Never** run `DROP DATABASE` , `TRUNCATE customers` , or a full replace import on a database you mean to preserve.

2D · Knowing what you already ran

- Keep a simple text note: “Live DB — ran through 06b on 2026-05-01” — update it when you apply a new file.
- In phpMyAdmin → **Structure**: check if a table exists (e.g. `sales_credit_notes` → if yes, 07 is likely done).
- For a new column: click table **Structure** and see if the column name appears (e.g. `account_customer` on `customers` → 06a applied).

2E · If PHP shows “unknown column” or “table doesn’t exist”

- Match the error to the feature (credit notes → 07 ; shop messages → 06e ; AR account flags → 06a).
- Run the **missing** script for that database. If dependency errors appear (missing parent table), run the **earlier** script in the order above first.

QUICK COMPARISON

	Path 1 Full replace	Path 2 Incremental
--	---------------------	--------------------

Keeps existing rows	No — unless backup restored selectively	Yes — adds/changes structure only
Main tool	Empty DB + Import dump <i>or</i> full script chain + seed	Paste missing <code>sql/*.sql</code> files only
Risk	Losing live-only data	Low if scripts are additive; read errors before retry

COPYING “ONLY SOME DATA” BETWEEN SERVERS (ADVANCED)

If you must merge environments without full wipe: export **specific tables** from phpMyAdmin (**Custom** export, tick chosen tables), then import carefully into the target — watch **foreign keys** and **ID clashes**. This is expert territory; default recommendation for Autowagen is Path 2 migrations plus normal backups.